



Instituto Politécnico Nacional

Semestre A26

Algoritmos



Alumno: **González Mendoza Juan Pablo**

Fecha: **Junio, 2026**

Tarea 4 **Práctica Árboles.**

Un árbol binario es una estructura de datos jerárquica en la que cada nodo tiene a lo más dos hijos: izquierdo y derecho. En la práctica existen distintas formas de almacenar un árbol en memoria, y la elección entre ellas depende del tipo de árbol y de las operaciones que se necesiten realizar. En esta práctica se implementan dos representaciones tabulares de un árbol binario de búsqueda: la opción 1 es una tabla con nodos y apuntadores, donde cada nodo es una estructura que contiene el dato y dos punteros explícitos a sus hijos; la opción 2 es una tabla con hijo izquierdo y derecho, que utiliza arreglos paralelos donde los índices de los hijos reemplazan a los apuntadores. Adicionalmente se incluye una variante implícita (opción 2-B) donde las relaciones padre-hijo se calculan mediante fórmulas aritméticas sobre los índices del arreglo.

Marco Teórico

En cuanto al marco teórico, la opción 1 (nodos con apuntadores) es la representación más común y flexible. Cada nodo se define como una estructura:

$$\text{Nodo} = \{\text{dato}, \text{izq}, \text{der}\}$$

donde *izq* y *der* son punteros a otros nodos o NULL si no existe el hijo. La memoria se asigna dinámicamente y el árbol puede crecer sin límite predefinido. Entre sus ventajas están el tamaño dinámico y una inserción y eliminación eficientes; como desventaja, cada nodo requiere espacio adicional para dos punteros (16 bytes en arquitecturas de 64 bits).

La opción 2 (tabla con índices explícitos), en lugar de punteros, emplea arreglos paralelos: `datos[]`, `hijoIzq[]`, `hijoDer[]` y opcionalmente `padre[]`. Los hijos de un nodo en posición i se almacenan como los índices `hijoIzq[i]` y `hijoDer[i]` dentro de los mismos arreglos. Sus ventajas son que no requiere asignación dinámica y tiene buena localidad de caché; sus desventajas son el tamaño máximo fijo y el manejo manual de índices.

La opción 2-B (arreglo implícito con fórmulas de índices) permite, para árboles completos, prescindir por completo de punteros e índices explícitos. Si un nodo está en la posición i (indexación base 0), entonces:

$$\text{Hijo izquierdo} = 2i + 1, \quad \text{Hijo derecho} = 2i + 2, \quad \text{Padre} = \left\lfloor \frac{i-1}{2} \right\rfloor.$$

Esta representación es la base de las colas de prioridad implementadas como *heaps*.

Pseudocódigos

Algorithm 1: Inserción BST — Opción 1 (Nodos y Apuntadores)

```

1 Función InsertarBST(raiz, valor)
2   if raiz = NULL then
3     |   return CrearNodo(valor)
4   end
5   if valor < raiz.dato then
6     |   raiz.izq ← InsertarBST(raiz.izq, valor)
7   end
8   else if valor > raiz.dato then
9     |   raiz.der ← InsertarBST(raiz.der, valor)
10  end
11  return raiz

```

Algorithm 2: Mostrar Tabla por Niveles — Opción 1

Input: Raíz del árbol

Output: Tabla con dato, dirección del nodo, hijo izq y hijo der

```
1 cola  $\leftarrow$  cola vacía
2 Encolar raiz
3 idx  $\leftarrow$  0
4 while cola no vacía do
5   actual  $\leftarrow$  Desencolar
6   Imprimir idx, actual.dato, dirección de actual, dirección de actual.izq, dirección de
     actual.der
7   if actual.izq  $\neq$  NULL then
8     | Encolar actual.izq
9   end
10  if actual.der  $\neq$  NULL then
11    | Encolar actual.der
12  end
13  idx  $\leftarrow$  idx + 1
14 end
```

Algorithm 3: Inserción BST — Opción 2 (Tabla con Índices)

Input: Arreglos $\text{datos}[]$, $\text{hijoIzq}[]$, $\text{hijoDer}[]$, $\text{padre}[]$, contador n , valor a insertar v

```
1  $\text{datos}[n] \leftarrow v$ 
2 if  $n = 0$  then
3    $n \leftarrow n + 1$ 
4   return // es la raíz
5 end
6  $\text{actual} \leftarrow 0$ 
7 while true do
8   if  $v < \text{datos}[\text{actual}]$  then
9     if  $\text{hijoIzq}[\text{actual}] = \text{VACIO}$  then
10       $\text{hijoIzq}[\text{actual}] \leftarrow n$ 
11       $\text{padre}[n] \leftarrow \text{actual}$ 
12      break
13    end
14     $\text{actual} \leftarrow \text{hijoIzq}[\text{actual}]$ 
15  end
16  else
17    if  $\text{hijoDer}[\text{actual}] = \text{VACIO}$  then
18       $\text{hijoDer}[\text{actual}] \leftarrow n$ 
19       $\text{padre}[n] \leftarrow \text{actual}$ 
20      break
21    end
22     $\text{actual} \leftarrow \text{hijoDer}[\text{actual}]$ 
23  end
24 end
25  $n \leftarrow n + 1$ 
```

Algorithm 4: Inserción BST — Opción 2-B (Arreglo Implícito)

Input: Arreglo $arr[]$, valor a insertar v

```
1 if  $arr[0] = VACIO$  then
2   |  $arr[0] \leftarrow v$ 
3   | return
4 end
5  $i \leftarrow 0$ 
6 while  $i < capacidad$  do
7   | if  $arr[i] = VACIO$  then
8     |  $arr[i] \leftarrow v$ 
9     | return
10  | end
11  | if  $v < arr[i]$  then
12    |  $i \leftarrow 2i + 1$                                 // hijo izquierdo
13  | end
14  | else
15    |  $i \leftarrow 2i + 2$                                 // hijo derecho
16  | end
17 end
```

Algorithm 5: Recorridos del Árbol Binario

```
1 Función Inorden( $nodo$ )
2   | if  $nodo = NULL$  then
3     | return
4   | end
5   | Inorden( $nodo.izq$ )
6   | Imprimir  $nodo.dato$ 
7   | Inorden( $nodo.der$ )

8 Función Preorden( $nodo$ )
9   | if  $nodo = NULL$  then
10  | | return
11  | end
12  | Imprimir  $nodo.dato$ 
13  | Preorden( $nodo.izq$ )
14  | Preorden( $nodo.der$ )

15 Función Postorden( $nodo$ )
16  | if  $nodo = NULL$  then
17  | | return
18  | end
19  | Postorden( $nodo.izq$ )
20  | Postorden( $nodo.der$ )
21  | Imprimir  $nodo.dato$ 
```

Comparación de Representaciones

En la comparación de representaciones se observa que la memoria por nodo es $d + 2p$ en la opción 1 (donde d es el tamaño del dato y p el de un puntero, 8 bytes en x86-64), $4d$ en la opción 2, y solo d en la opción 2-B. En cuanto al tamaño, la opción 1 es dinámica, mientras que las opciones 2 y 2-B tienen tamaño fijo. El acceso al padre requiere un puntero extra en la opción 1, es $O(1)$ con arreglo en la opción 2, y $O(1)$ con fórmula en la opción 2-B. La opción 1 es ideal para árboles generales, la opción 2 para árboles con n conocido, y la opción 2-B para árboles completos o heaps.

Característica	Op. 1 (Ptrs)	Op. 2 (Índices)	Op. 2-B (Implícito)
Memoria por nodo	$d + 2p^*$	$4d$	d
Tamaño dinámico	Sí	No (fijo)	No (fijo)
Acceso al padre	Requiere puntero extra	$O(1)$ con arreglo	$O(1)$ con fórmula
Ideal para	Árboles generales	Árboles con n conocido	Árboles completos / Heaps

* d = tamaño del dato, p = tamaño de un puntero (8 bytes en x86-64).

Resultados

Se insertaron los valores $\{50, 30, 70, 20, 40, 60, 80, 10, 25, 35\}$ en un BST. Las tres representaciones producen los mismos recorridos, los cuales se detallan en la siguiente tabla:

Tipo de Recorrido	Secuencia de Valores
En orden (ordenado)	10, 20, 25, 30, 35, 40, 50, 60, 70, 80
Preorden	50, 30, 20, 10, 25, 40, 35, 70, 60, 80
Postorden	10, 25, 20, 35, 40, 30, 60, 80, 70, 50

La opción 1 utiliza 240 bytes (24 por nodo), la opción 2 utiliza 160 bytes (16 por nodo) y la opción 2-B utiliza únicamente 40 bytes (4 por nodo), confirmando que la representación implícita es la más eficiente en memoria para árboles completos.

Conclusión

Finalmente, ambas representaciones tabulares son alternativas válidas a los apuntadores tradicionales para implementar árboles binarios. La tabla con índices explícitos elimina la sobrecarga de punteros y ofrece buena localidad de caché, así como la representación implícita basada en fórmulas aritméticas, que es la más compacta y constituye la base de estructuras fundamentales como el heap binario. La elección entre una u otra depende de si el árbol es completo, si se conoce el número máximo de nodos y de las operaciones requeridas.